

SCHEDULING ALGORITHMS

```
#include <stdio.h>

int main() {
    int n, i;
    int bt[20], wt[20], tat[20];

    printf("Enter number of processes: ");
    scanf("%d", &n);

    printf("Enter burst time for each process:\n");

    for (i = 0; i < n; i++) {
        printf("P%d: ", i + 1);
        scanf("%d", &bt[i]);
    }

    // Waiting time
    wt[0] = 0;

    for (i = 1; i < n; i++) {
        wt[i] = wt[i - 1] + bt[i - 1];
    }

    // Turnaround time
    for (i = 0; i < n; i++) {
        tat[i] = wt[i] + bt[i];
    }

    printf("\nProcess\tBT\tWT\tTAT\n");

    for (i = 0; i < n; i++) {
        printf("P%d\t%d\t%d\t%d\n",
            i + 1, bt[i], wt[i], tat[i]);
    }

    return 0;
}
```

```
sai_pavani@Sai-Pavani:~/OS_LAB/Scheduling_Algorithms$ ./fcfs_scheduling
Enter number of process: 3
Enter burst time for each process:
Enter process 1: 4
Enter process 2: 2
Enter process 3: 3

Process BT      WT      TAT
1       4       0       4
2       2       4       6
3       3       6       9
```

```

#include <stdio.h>

int main() {
    int n, i, j;
    int bt[20], wt[20], tat[20];
    int temp;

    printf("Enter number of processes: ");
    scanf("%d", &n);

    printf("Enter burst time:\n");

    for (i = 0; i < n; i++) {
        printf("P%d: ", i + 1);
        scanf("%d", &bt[i]);
    }

    // Sort burst times
    for (i = 0; i < n - 1; i++) {
        for (j = i + 1; j < n; j++) {
            if (bt[i] > bt[j]) {
                temp = bt[i];
                bt[i] = bt[j];
                bt[j] = temp;
            }
        }
    }

    // Waiting time
    wt[0] = 0;

    for (i = 1; i < n; i++) {
        wt[i] = wt[i - 1] + bt[i - 1];
    }

    // Turnaround time
    for (i = 0; i < n; i++) {
        tat[i] = wt[i] + bt[i];
    }

    printf("\nProcess\tBT\tWT\tTAT\n");

    for (i = 0; i < n; i++) {
        printf("P%d\t%d\t%d\t%d\n",
            i + 1, bt[i], wt[i], tat[i]);
    }

    return 0;
}

```

```

sai_pavani@Sai-Pavani:~/OS_LAB/Scheduling_Algorithms$ nano SJF_NONpreemptive.c
sai_pavani@Sai-Pavani:~/OS_LAB/Scheduling_Algorithms$ gcc SJF_NONpreemptive.c -o SJF_NONpreemptive
sai_pavani@Sai-Pavani:~/OS_LAB/Scheduling_Algorithms$ ./SJF_NONpreemptive

```

Enter number of processes: 5

Enter burst time:

P1: 4

P2: 3

P3: 1

P4: 6

P5: 2

Process	BT	WT	TAT
P1	1	0	1
P2	2	1	3
P3	3	3	6
P4	4	6	10
P5	6	10	16

```

#include <stdio.h>
int main() {

    int n, i, j;

    int bt[20], priority[20], wt[20], tat[20];

    int temp;

    printf("Enter number of processes: ");
    scanf("%d", &n);

    printf("Enter burst time and priority:\n");
    for (i = 0; i < n; i++) {

        printf("P%d Burst Time: ", i + 1);

        scanf("%d", &bt[i]);

        printf("P%d Priority: ", i + 1);

        scanf("%d", &priority[i]);

    }

    // Sort according to priority

```

```

// Sort according to priority
for (i = 0; i < n - 1; i++) {

    for (j = i + 1; j < n; j++) {

        if (priority[i] > priority[j]) {

            temp = priority[i];

            priority[i] = priority[j];

            priority[j] = temp;

            temp = bt[i];

            bt[i] = bt[j];

            bt[j] = temp;

        }

    }

}

// Waiting time
wt[0] = 0;

```

```

// Waiting time
wt[0] = 0;

for (i = 1; i < n; i++) {
    wt[i] = wt[i - 1] + bt[i - 1];
}

// Turnaround time
for (i = 0; i < n; i++) {
    tat[i] = wt[i] + bt[i];
}

printf("\nProcess\tBT\tPriority\tWT\tTAT\n");

for (i = 0; i < n; i++) {
    printf("P%d\t%d\t%d\t\t%d\t%d\n",
        i + 1, bt[i], priority[i], wt[i], tat[i]);
}

return 0;
}

```

```

sai_pavani@Sai-Pavani:~/OS_LAB/Scheduling_Algorithms$ nano priority.c
sai_pavani@Sai-Pavani:~/OS_LAB/Scheduling_Algorithms$ gcc priority.c -o priority
sai_pavani@Sai-Pavani:~/OS_LAB/Scheduling_Algorithms$ ./priority
Enter number of processes: 4
Enter burst time and priority:
P1 Burst Time: 2
P1 Priority: 3
P2 Burst Time: 1
P2 Priority: 4
P3 Burst Time: 6
P3 Priority: 5
P4 Burst Time: 3
P4 Priority: 1

Process BT      Priority      WT      TAT
P1      3        1          0        3
P2      2        3          3        5
P3      1        4          5        6
P4      6        5          6       12

```

```
#include <stdio.h>

int main() {
    int n, i;
    int bt[20], rem_bt[20];
    int wt[20], tat[20];
    int quantum;
    int time = 0;
    int done;

    printf("Enter number of processes: ");
    scanf("%d", &n);

    printf("Enter burst time:\n");

    for (i = 0; i < n; i++) {
        printf("P%d: ", i + 1);
        scanf("%d", &bt[i]);
        rem_bt[i] = bt[i];
    }
```

```
        wt[i] = 0;
    }

    printf("Enter time quantum: ");
    scanf("%d", &quantum);
    // Round Robin
    while (1) {
        done = 1;

        for (i = 0; i < n; i++) {
            if (rem_bt[i] > 0) {
                done = 0;

                if (rem_bt[i] > quantum) {
                    time = time + quantum;
                    rem_bt[i] = rem_bt[i] - quantum;
                }
            }
        }
    }
```

```

        else {
            time = time + rem_bt[i];
            wt[i] = time - bt[i];
            rem_bt[i] = 0;
        }
    }

    if (done == 1)
        break;
}

// Turnaround time
for (i = 0; i < n; i++) {
    tat[i] = wt[i] + bt[i];
}

```

```

// Turnaround time

for (i = 0; i < n; i++) {
    tat[i] = wt[i] + bt[i];
}

printf("\nProcess\tBT\tWT\tTAT\n");

for (i = 0; i < n; i++) {
    printf("P%d\t%d\t%d\t%d\n",
           i + 1, bt[i], wt[i], tat[i]);
}

return 0;
}

```

```
sai_pavani@Sai-Pavani:~/OS_LAB/Scheduling_Algorithms$ nano ROUND_ROBIN.c
sai_pavani@Sai-Pavani:~/OS_LAB/Scheduling_Algorithms$ gcc ROUND_ROBIN.c -o ROUND_ROBIN
sai_pavani@Sai-Pavani:~/OS_LAB/Scheduling_Algorithms$ ./ROUND_ROBIN
Enter number of processes: 4
Enter burst time:
P1: 2
P2: 1
P3: 4
P4: 5
Enter time quantum: 3

Process BT      WT      TAT
P1      2      0      2
P2      1      2      3
P3      4      6     10
P4      5      7     12
sai_pavani@Sai-Pavani:~/OS_LAB/Scheduling_Algorithms$ |
```